Assignment Instructions

Listen

CIS 228 Introduction to JavaScript - Assignment: Convenience Store Inventory Manager

Assignment Overview

In this assignment, you will build a simplified inventory management system for a convenience store using JavaScript **Collections** (arrays of objects). You'll practice essential array methods—map(), filter(), and reduce()—to manipulate and analyze grocery item data.
This assignment is based on the JavaScript Collections demonstration but adapted for a convenience store context with grocery items instead of computer parts.

Learning Objectives

By completing this assignment, you will demonstrate the ability to: - Create and manipulate a collection (array of objects) representing grocery items - Use map() to transform collection data - Use filter() to select specific subsets of data - Use reduce() to calculate aggregate values - Display results in the browser console and/or DOM

Requirements

1. Data Structure

Create a collection called groceryItems containing **at least 8 grocery items** from a convenience store. Each item must be an object with these properties: - id (number): Unique identifier - name (string): Product name - category (string): Category like "beverages", "snacks", "dairy", "bakery", "canned", "frozen" - price (number): Price in dollars (e.g., 2.99) - inStock (boolean): Availability status - quantity (number): Number of units in stock

2. Required Operations

Implement the following operations using the specified array methods:

A. Map Operations

1. **Extract all item names** - Create an array containing only the names of all grocery items
2. **Create price tags** - Create an array of formatted strings like "$2.99 - Milk"
3. **Calculate total value per item** - Create an array of objects with name and totalValue (price × quantity)

B. Filter Operations

1. **In-stock items only** - Filter to show only items where inStock is true

2. **Items by category** - Filter items belonging to a specific category (e.g., “beverages”)
3. **Low stock items** - Filter items with quantity less than 5
4. **Expensive items** - Filter items with price greater than $5.00

C. Reduce Operations

1. **Total inventory value** - Calculate the sum of (price × quantity) for all items
2. **Total value by category** - Calculate total inventory value for a specific category
3. **Count of in-stock items** - Count how many items are currently in stock
4. **Average price** - Calculate the average price of all items

3. Output Requirements

Display all results in the browser console using console.log() with clear labels. Format your output similar to:

```
=== CONVENIENCE STORE INVENTORY REPORT ===

--- MAP OPERATIONS ---
All Item Names: [...]
Price Tags: [...]
Item Total Values: [...]

--- FILTER OPERATIONS ---
In-Stock Items: [...]
Beverages: [...]
Low Stock Items (< 5): [...]
Expensive Items (> $5): [...]

--- REDUCE OPERATIONS ---
Total Inventory Value: $XXX.XX
Beverages Total Value: $XX.XX
In-Stock Item Count: X
Average Price: $X.XX
```

Starter Code Template

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Convenience Store Inventory</title>
</head>
<body>
    <h1>Convenience Store Inventory Manager</h1>
    <p>Open the browser console (F12) to see the inventory report.</p>
    <script src="inventory.js"></script>
</body>
</html>
// inventory.js - Your implementation goes here

// 1. CREATE YOUR GROCERY ITEMS COLLECTION (minimum 8 items)
// const groceryItems = [
//     { id: 1, name: "...", category: "...", price: 0.00, inStock: true, quantity: 0
},
//     // ... add at least 7 more items
// ];

// 2. MAP OPERATIONS
// const itemNames = groceryItems.map(...);
// const priceTags = groceryItems.map(...);
// const itemTotalValues = groceryItems.map(...);

// 3. FILTER OPERATIONS
// const inStockItems = groceryItems.filter(...);
// const beverages = groceryItems.filter(...);
// const lowStockItems = groceryItems.filter(...);
// const expensiveItems = groceryItems.filter(...);

// 4. REDUCE OPERATIONS
// const totalInventoryValue = groceryItems.reduce(...);
// const beveragesTotalValue = groceryItems.filter(...).reduce(...);
// const inStockCount = groceryItems.reduce(...);
// const averagePrice = groceryItems.reduce(...);

// 5. DISPLAY RESULTS
// console.log("=== CONVENIENCE STORE INVENTORY REPORT ===");
// ... your console.log statements
```

Hints & Implementation Guidance

Map Hints

```
// Basic map syntax: array.map(item => transformation)
// Extract names:
const names = groceryItems.map(item => item.name);

// Create formatted strings:
const priceTags = groceryItems.map(item => `$${item.price.toFixed(2)} -
${item.name}`);

// Create new objects:
const values = groceryItems.map(item => ({
    name: item.name,
    totalValue: item.price * item.quantity
}));
```

Filter Hints

```
// Basic filter syntax: array.filter(item => condition)
// Boolean property:
const inStock = groceryItems.filter(item => item.inStock);

// String comparison:
const beverages = groceryItems.filter(item => item.category === "beverages");

// Numeric comparison:
const lowStock = groceryItems.filter(item => item.quantity < 5);
const expensive = groceryItems.filter(item => item.price > 5.00);
```

Reduce Hints

```
// Basic reduce syntax: array.reduce((accumulator, current) => newAccumulator,
initialValue)

// Sum of price * quantity:
const totalValue = groceryItems.reduce((sum, item) => sum + (item.price *
item.quantity), 0);

// Count with condition:
const inStockCount = groceryItems.reduce((count, item) => count + (item.inStock ? 1 :
0), 0);

// Average (requires two steps or single reduce with object):
const priceSum = groceryItems.reduce((sum, item) => sum + item.price, 0);
const avgPrice = priceSum / groceryItems.length;

// Or single reduce for average:
const avgResult = groceryItems.reduce(
    (acc, item) => ({ sum: acc.sum + item.price, count: acc.count + 1 }),
    { sum: 0, count: 0 }
);
const averagePrice = avgResult.sum / avgResult.count;
```

Chaining Methods

You can chain methods together:

```
// Filter THEN reduce:
const beveragesValue = groceryItems
    .filter(item => item.category === "beverages")
    .reduce((sum, item) => sum + (item.price * item.quantity), 0);
```

Sample Grocery Items for Testing

Feel free to use or adapt these sample items:

ID	Name	Category	Price	In Stock	Quantity
1	Bottled Water	beverages	1.49	true	24
2	Potato Chips	snacks	2.99	true	12
3	Chocolate Bar	snacks	1.79	true	30

4	Whole Milk	dairy	3.99	true	8
5	White Bread	bakery	2.49	false	0
6	Canned Soup	canned	1.99	true	15
7	Frozen Pizza	frozen	6.99	true	6
8	Energy Drink	beverages	3.49	true	10
9	Yogurt	dairy	1.29	true	20
10	Gum Pack	snacks	0.99	true	50

Submission Requirements

Submit the following files: 1. index.html - Your HTML file 2. inventory.js - Your JavaScript implementation

Grading Rubric

Criteria	Points
Collection created with 8+ items, all required properties	20
Map operations correctly implemented (3 operations)	20
Filter operations correctly implemented (4 operations)	20
Reduce operations correctly implemented (4 operations)	20
Console output formatted clearly with labels	10
Code organization, comments, and readability	10
Total	**100**

Bonus Challenge (Optional - Extra 10 Points)

Add a feature to **find the most expensive item per category** using a combination of filter, reduce, or other array methods. Display the results in the console.

Good luck! Remember to test your code frequently by opening the browser console.

Submission

Click to add text

Word count: 0

Drag files here

Maximum size: 512 MB

or

Upload a file | Upload from cloud

Save and Close | Submit